



5-5 指令流水线

1. 指令流水线基本构成

- 五个基本流水段
 - IF 取指令：根据 PC 的值从存储器取出指令。
 - ID 指令译码/读寄存器：产生控制信号，同时从寄存器中取数据。
 - EX 执行/计算地址：执行运算操作或计算访存地址。
 - MEM 访存：对存储器进行读或写操作。
 - WB 写回：将操作结果写入寄存器。
- 理想流水线效果
 - 启动阶段：前几拍逐步填满流水线。
 - 稳定阶段：理想情况下，每个时钟周期都有一条指令进入流水线，也有一条指令完成。
 - 收尾阶段：最后几条指令依次流出流水线。
 - k 段流水线执行 n 条指令：理想情况下大约需要 $k + n - 1$ 个时钟周期。
 - 五段流水线：执行 n 条指令理想需要 $n + 4$ 个时钟周期。
- 数据通路设计原则
 - 流水段个数：以最复杂指令所用的功能段个数为准。
 - 流水段长度：以最复杂功能段的操作时间为准。
 - 时钟周期：由最慢流水段延迟决定，还要加上流水段寄存器延迟。
- 流水段内部结构
 - 组合逻辑：完成本阶段处理工作。
 - 流水段寄存器：保存本阶段组合逻辑处理结果，供下一阶段使用。
 - 基本形式： $\text{组合逻辑} + \text{寄存器}$ 。

2. 适合流水线的指令集特征

- 指令长度尽量一致
 - 作用：简化取指令操作。
 - 原因：若指令长度不等，取指令部件更复杂，也不利于译码。
- 指令格式尽量规整
 - 作用：简化指令译码。
 - 关键点：尽量保证源操作数寄存器位置相同。
 - 好处：在指令尚未完全译码时，就可尽早读取寄存器操作数。
- 采用 Load/Store 型指令风格
 - 含义：只有 Load 和 Store 指令能访问存储器，其他指令不能直接访存。
 - Load：从存储单元读出数据并装入寄存器，例如 MIPS 的 lw。
 - Store：把寄存器内容写入存储单元。
 - 好处：使 Load/Store 的地址计算和普通运算指令的执行步骤规整在同一个流水段中。
- 数据和指令对齐存放
 - 含义：数据和指令在存储器中按边界对齐。
 - 好处：所需数据可在一个流水段内从存储器得到。

3. 指令流水线实现

- 五个功能段
 - Ifetch：组合逻辑 A 中有指令存储器，实现取指令功能。
 - Reg/Dec：组合逻辑 B 中有寄存器堆，实现读寄存器和指令译码。
 - ALU：组合逻辑 C 中有 ALU，实现算术和逻辑运算。
 - Mem：组合逻辑 D 中有数据存储器，实现存储器读写。
 - Write：组合逻辑 E 中有通路选择器，选择写回寄存器的结果，并保证寄存器输入稳定。
- R 型指令
 - 功能：对 rs 和 rt 内容进行 ALU 运算，结果写入目的寄存器 rd。
 - 阶段划分：
 - IF：取指。
 - ID：译码并读取 rs、rt。
 - EX：ALU 运算。
 - MEM：空操作。
 - WB：ALU 结果写回 rd。
- I 型立即数运算指令
 - 功能：扩展 imm16，与 rs 内容运算，结果写入 rt。
 - 阶段划分：
 - IF：取指。
 - ID：译码并读取 rs。
 - EX：立即数扩展后参与 ALU 运算。
 - MEM：空操作。
 - WB：ALU 结果写回 rt。
- lw 指令
 - 功能： $R[rt] \leftarrow M[R[rs] + \text{SignExt}(\text{imm16})]$ 。
 - 阶段划分：
 - IF：取指。
 - ID：译码并读取 rs。
 - EX：ALU 计算存储单元地址。
 - MEM：从数据存储器读出数据。
 - WB：把读出的数据写入 rt。
- beq 指令
 - 功能：若 $R[rs] == R[rt]$ ，则转移。
 - 阶段划分：
 - IF：取指。
 - ID：译码并读取 rs、rt。
 - EX：对 $R[rs]$ 和 $R[rt]$ 做减法，同时计算目标地址 $PC + 4 + 4 \times \text{imm16}$ 。
 - MEM：根据比较结果决定是否把目标地址送 PC。
 - WB：空操作。

4. 流水线冒险

- 基本概念
 - 流水线冒险：实际执行中，某些情况会使流水线无法正确、按时执行后续指令，从而造成阻塞或停顿。
 - 主要类型：结构冒险、数据冒险、控制冒险。

5. 结构冒险

- 含义
 - 结构冒险：硬件资源冲突。
 - 产生原因：同一个硬件部件在同一时刻被不同指令同时使用。
- 解决策略
 - 规定使用时机：一个部件每条指令只能使用一次，并且只能在特定阶段使用。
 - 增加独立部件：通过设置多个独立部件避免资源冲突。

6. 数据冒险

- 含义
 - 数据冒险：也叫数据相关。
 - 产生原因：后面指令要使用前面指令的运算结果，但前面指令结果尚未产生或尚未写回。
- 解决策略
 - 插入 nop：由编译器在相关指令之间加入空操作指令。
 - 硬件阻塞：硬件检测到相关后，让流水线暂停若干周期。
 - 数据转发/旁路：一旦运算结果产生或存储器读出数据，就通过旁路直接送到后续指令 ALU 阶段输入端。

7. Load-use 冒险

- 含义
 - Load-use 冒险：前一条指令是 Load，并把数据写入寄存器；后一条指令马上使用该寄存器数据。
- 是否使用旁路
 - 不使用旁路：后一条指令的 ID 阶段需要等到 Load 的 WB 后才能执行。
 - 使用旁路：可把 MEM 阶段得到的结果送入后续指令 EX 阶段。
 - 仍需停顿：即使使用旁路，Load-use 场景通常仍需要插入一个时钟周期阻塞。
- 检测与处理
 - 检测阶段：在 ID 阶段检测 Load-use 冒险。
 - 停顿动作：检测到后，使 Load 后的相关指令及后续取指停顿一个时钟周期。
 - 控制信号清零：把 ID/EX 流水段的控制信号清零，不把译码得到的控制信号送给 EX。
 - 保持 IF/ID：保持 IF/ID 流水段寄存器的值不变。
 - 保持 PC：保持 PC 值不变，让阻塞后对应指令能重新取指。

8. 控制冒险

- 含义
 - 控制冒险：遇到改变指令执行顺序的情况时，流水线中指令的正常执行被阻塞。
 - 别名：由分支指令引起的控制冒险也称分支冒险。
- 分支成立时的处理
 - 冲刷流水线：如果跳转成立，已经进入流水线但不该执行的后续指令要被冲刷掉。
 - 延迟损失时间片：由于流水线阻塞带来的延迟执行周期数量。

9. 流水线中的多发技术

- 超标量技术
 - 基本思想：每个时钟周期内可同时并发多条独立指令。
 - 执行方式：以并行操作方式将两条或两条以上指令编译并执行。
 - 硬件要求：多个功能部件、多个指令译码电路、多个寄存器端口和多条总线。
 - 编译要求：编译程序决定哪些相邻指令可以并行执行。
 - 理想效果：理想状态下，采用超标量流水线技术的 CPU，CPI 可以小于 1。
- 超流水线技术
 - 基本思想：在流水线中插入更多流水线寄存器，相当于把流水线继续细分。
 - 效果：处理器周期比普通流水线更短。
 - 限制：硬件不能自动调整指令执行顺序，仍依赖编译程序优化。
 - 对比：它提高的是时钟频率/阶段细分程度，不一定每拍发射多条指令。
- 超长指令字技术 VLIW
 - 基本思想：由编译程序在编译时挖掘指令间潜在并行性。
 - 执行方式：把多条可并行指令组合成一条具有多个操作码字段的超长指令。
 - 优势：并行处理能力高。
 - 代价：对优化编译器要求更高，对 Cache 容量要求更大。